Simplify, Order, Break, Repeat

Markus Anders
RPTU Kaiserslautern-Landau, Germany

Cayden Codel
Carnegie Mellon University, Pittsburgh, PA, USA

Marijn J. H. Heule
Carnegie Mellon University, Pittsburgh, PA, USA

Abstract

Existing symmetry-breaking techniques for SAT constrain the search space without simplifying the formula. Lex-leader predicates, the most widely used approach, add global ordering constraints to remove symmetric solutions, but often at the cost of substantial formula blowup and large proofs. Moreover, they ignore structural properties of the formula, such as connectedness, that could enable stronger symmetry breaking. In this paper, we present a new algorithm that treats symmetry breaking not merely as a restriction mechanism, but as a simplification mechanism. Notably, it only adds unit and binary symmetry-breaking clauses, which enables strong formula simplifications. This can expose additional symmetries and in turn allows for multiple rounds of symmetry breaking. Crucially, we exploit connectivity in the formula's graph representation, including the presence of cliques, to guide the order of symmetry breaking. We implemented our algorithm in the SATSUMA tool and evaluated it on a large set of benchmarks. As a preprocessing step for CaDiCaL, it improves PAR-2 scores by 22% on the SAT competition 2025 and by 12% on the SAT anniversary track. It also substantially outperforms lex-leader and orbitopal fixing while producing compact proofs.

2012 ACM Subject Classification Theory of computation → Automated reasoning

Keywords and phrases Boolean satisfiability, symmetry handling, preprocessing, Schreier-Sims

Digital Object Identifier 10.4230/LIPIcs.SAT.2026.4

Supplementary Material *Software (Source code)*: `https://doi.org/10.5281/zenodo.20185023`

Funding This work was supported by the National Science Foundation (NSF) under grant DMS-2434625 and by the AFRL and DARPA under Agreement FA8750-24-9-1000.

Acknowledgements We thank Annika Jäger, Christopher Hojny, and Marc Pfetsch for their insights into related work in optimization. This work substantially benefited from Dagstuhl Seminar 25231.

1 Introduction

Symmetries in Boolean satisfiability (SAT) instances can cause solvers to redundantly explore equivalent regions of the search space, leading to significant performance degradation. *Symmetry breaking* mitigates this problem by restricting the search space to asymmetric solutions. This technique is effective on many highly symmetric benchmarks [1, 4, 17, 18], and it has been integrated into a wide range of solver paradigms [21, 36]. Yet despite these successes, symmetry breaking has not become a standard component of modern SAT solvers.

The difficulty is largely algorithmic. Effective symmetry handling must balance pruning strength against computational overhead, all without disrupting core solver heuristics. Moreover, every symmetry-breaking inference must be justified by a formally checkable proof [30], which constrains the kinds of symmetry reasoning that are admissible.

Today, the most practical SAT symmetry-breaking tools add *lex-leader constraints* to the formula, justified via the so-called dominance rule [3, 13]. This approach, however, suffers from two significant drawbacks. First, the dominance rule is only supported by systems based

© Markus Anders, Cayden Codel, and Marijn J. H. Heule;
licensed under Creative Commons License CC-BY 4.0

on pseudo-Boolean reasoning, which empirically incur substantially higher proof generation and verification costs than standard clausal proofs. Second, lex-leader constraints tend to blow up the size of the formula, potentially offsetting their pruning benefits.

Motivated by these shortcomings, recent work [5] introduced a new symmetry-breaking approach for SAT inspired by *orbitopal fixing* from mixed-integer programming [27]. This method adds only unit clauses to the formula, which avoids formula blowup, and it admits succinct proofs entirely within the substitution redundancy (SR) clausal proof system [16, 22], which integrates naturally with existing SAT proof infrastructures. These advantages, however, come at the cost of reduced pruning power: in its current form, orbitopal fixing is structurally more limited than lex-leader constraints and requires certain cardinality constraints to be present in the formula, restricting its applicability.

This state of affairs reveals a fundamental tension at the heart of certified symmetry breaking in SAT: stronger pruning demands more expressive and more expensive proof systems, whereas the efficiency of clausal proofs currently means using weaker symmetry techniques. In this work, we investigate how far symmetry breaking can be pushed within the confines of the SR clausal proof system.

Contribution. We present a new symmetry-breaking algorithm for certified SAT solving that combines ideas from orbitopal fixing [5], recent work on iterated Schreier-Sims table cuts [24], and standard CNF preprocessing techniques. At a high level, it looks like this:

1. Simplify the formula using CNF preprocessing techniques.
2. Compute the symmetries of the formula. If only trivial symmetries remain, terminate.
3. Break detected symmetries by adding *unit* and *binary clauses*.
4. Repeat from Step 1.

Unlike lex-leader approaches, which add long structured encodings and new variables to the formula, our algorithm produces a formula that is usually *smaller* than the input. It also emits succinct SR proofs justifying the clauses it added.

Our algorithm has three key features. First, because the algorithm only adds unit and binary clauses to the formula, the formula can be simplified using standard preprocessing techniques, which can in turn reveal new symmetries to break. Having multiple rounds of symmetry detection is unique among static symmetry-breaking techniques for SAT,[1] and is inspired by recent symmetry-handling techniques for stable set solvers [24]. Second, we take inspiration from successful graph coloring heuristics [23, 25] to guide the order in which we add symmetry-breaking clauses. In particular, we fix variables involved in *cliques* in the formula's variable-clause graph. To find these cliques, we use the maximum-clique solver CLIQUER [38], along with some lightweight heuristics. Third, our algorithm uses the Schreier-Sims algorithm [32, 34] to introduce symmetry-breaking clauses. Compared to more common filtering-based heuristics [4, 18], this approach *systematically* generates clauses for a chosen pointwise stabilizer chain while allowing for more flexible variable orderings.

Experimentally, our algorithm is quite effective. When implemented in the SATSUMA [4] symmetry-breaking tool and used as a preprocessing step for the state-of-the-art SAT solver CaDiCaL [11], our algorithm improves PAR-2 scores on benchmarks from the SAT anniversary track by 12% and the SAT Competition 2025 by 22%. It is also highly effective on a variety of crafted benchmarks, *completely solving* many of them during preprocessing. The preprocessing overhead is negligible, amounting to less than 1% of the average solve time. Overall, our algorithm outperforms previous work on orbitopal fixing symmetry breaking, as well as state-of-the-art lex-leader symmetry-breaking techniques.

[1] A partial exception is CryptoMiniSAT [35], which does symmetry breaking during inprocessing rounds.

2 Preliminaries

We assume that the reader is generally familiar with concepts from SAT solving. For a comprehensive introduction, please refer to the Handbook of Satisfiability [12].

The propositional formulas in this paper are all in conjunctive normal form (CNF), consisting of conjunctions of disjunctive clauses each containing Boolean literals. A literal ℓ is either a Boolean variable x or its negation $\overline{x}$.

We interpret clauses as sets of literals and formulas as sets of clauses, meaning we may, for example, write the clause $(x \vee \overline{y} \vee z)$ as $\{x, \overline{y}, z\}$. We write $\mathrm{Var}(F)$ and $\mathrm{Lit}(F)$ to refer to the variables and literals occurring in F, respectively. Two formulas F and F' are *equisatisfiable* if F is satisfiable iff F' is satisfiable.

Since cardinality constraints are used by both our algorithm and recent work on orbitopal fixing in SAT [5], we define a couple of relevant notions here.

First, a clause C is an *AMO clause* if all of its literals appear in a trivial *at-most-one* (AMO) encoding in a given CNF F. In other words, C is an AMO clause if $(\overline{\ell}_1 \vee \overline{\ell}_2) \in F$ for every pair of distinct literals $\ell_1, \ell_2 \in C$. As a result, if any literal in C is set to true, then all of the other literals in C are set to false under unit propagation.

Second, a clause $C \in F$ is a *unique literal clause* (ULC) with respect to F if none of its literals appear elsewhere in $F \setminus C$. For ULCs, the following property holds.

▶ **Lemma 1** ([33], Lemma 4). *Let F be a formula, and let $C \in F$ be a ULC. If F is satisfiable, then it can be satisfied with exactly one literal in C assigned to true.*

A corollary of Lemma 1 is that we can transform any ULC C into an AMO clause by adding the binary clauses $(\overline{\ell}_1 \vee \overline{\ell}_2)$ to F for every pair of distinct literals $\ell_1, \ell_2 \in C$. Notably, the addition of these clauses preserves the satisfiability of F.

2.1 Substitution Redundancy

All modern SAT solvers are *certifying algorithms* [30], meaning they produce answers that can be checked by formally-verified software. In the satisfiable case, the solver returns a truth assignment τ, and it is trivial to check whether τ satisfies the formula. The more interesting case is when the formula is unsatisfiable. Today, the de facto standard is for the solver to emit a proof of unsatisfiability in a *clausal proof system*. The most popular format is DRAT [37], based on the RAT proof system [26]. In this paper, we use the *substitution redundancy* (SR) proof system [15, 16, 22, 31], which is a generalization of RAT.

Clausal proofs comprise a series of addition and deletion steps. Each addition step adds a *redundant* clause C to the formula F, where C is redundant if F and $(F \wedge C)$ are equisatisfiable. Addition steps may also include a *witness* ω that helps show the redundancy of C. In SR, witnesses are substitutions $\omega : \mathrm{Var}(F) \to \mathrm{Lit}(F) \cup \{\top, \bot\}$ mapping variables to literals or to fixed truth values. Substitutions extend to literals in the natural way.

To show that a clause C is redundant, it suffices to show that it is *substitution redundant*.

▶ **Definition 2** (Substitution redundant [16]). *A clause C is* substitution redundant *for a formula F if there exists a witness ω such that $F \wedge \neg C \vdash_1 (F \wedge C)|_\omega$, where $\neg C \coloneqq \wedge_{\ell \in C} \overline{\ell}$ and where $F|_\omega$ means substituting each literal of F with its image under ω.*

Intuitively, the witness ω repairs any assignment τ that satisfies F but not C into an assignment that satisfies both. The SR rule uses unit propagation $\vdash_1$ rather than general entailment $\vDash$ because unit propagation is efficiently checkable by a proof checker.

Today, only the DSR/LSR [16] and VeriPB [22] proof formats support the SR rule. The implementation of our algorithm in SATSUMA can produce proofs in either format.

2.2 Symmetry in SAT

A symmetry of a formula F is a permutation of $\mathrm{Lit}(F)$ that maps F back to itself. Formally, let σ be a permutation of $\mathrm{Lit}(F)$, and let $\sigma(F)$ be the formula obtained by relabeling the literals of F under σ. Then σ is a *syntactic symmetry* of F if:

1. $\sigma(F) = F$, and
2. σ commutes with negation, i.e., $\neg\sigma(\ell) = \sigma(\bar{\ell})$ for all $\ell \in \mathrm{Lit}(F)$.

The set of all symmetries of F together with the composition operation $\circ$ forms a *permutation group*, denoted $\mathrm{Aut}(F)$. We write $\Gamma' \leq \Gamma$ if Γ' is a *subgroup* of a permutation group Γ, meaning that $\Gamma' \subseteq \Gamma$ and Γ' is also a permutation group.

In this paper, we use two standard group-theoretic notions: stabilizers and orbits. The *orbit* of a literal ℓ in a permutation group Γ is the set of literals reachable from ℓ using permutations of Γ. Formally, the orbit of ℓ in Γ is defined as $O_\Gamma(\ell) \coloneqq \{\sigma(\ell) \mid \sigma \in \Gamma\}$.

Given a set of literals L, the *pointwise stabilizer* $\Gamma_{(L)}$ is the set of permutations in Γ mapping every literal $\ell \in L$ to itself. Formally, we define:

$$\Gamma_{(L)} \coloneqq \{\sigma \in \Gamma \mid \sigma(\ell) = \ell \text{ for all } \ell \in L\}.$$

Observe that $\Gamma_{(L)} \leq \Gamma$ holds. If $L = \{\ell\}$ contains a single literal, we will write $\Gamma_{(\ell)}$.

2.3 Orbitopal Fixing

Orbitopal fixing [5, 9, 27] is a symmetry-breaking technique first studied by the mixed-integer programming community. It derives unit clauses from a matrix of literals containing row symmetry [19] and certain cardinality constraints. In the SAT setting, the derivation of these unit clauses is justified with short SR proofs [5]. In Section 4.3, we discuss how we optimized this technique as a sub-procedure for our preprocessing algorithm.

Orbitopal fixing applies when a subset of the literals of F can be arranged into an $n \times m$ matrix $M = (\ell_{i,j}) \subseteq \mathrm{Lit}(F)$ that satisfies the following two properties.

1. M exhibits *row symmetry*, where for all $i_1, i_2 \in [1, n]$, there exists a symmetry $\sigma_{i_1,i_2} \in \mathrm{Aut}(F)$ that swaps matrix rows i_1 and i_2. Formally:

$$\sigma_{i_1,i_2}(\ell) = \begin{cases} \ell_{i_2,j} & \text{if } \ell = \ell_{i_1,j} \text{ for some } j \in [1,m] \\ \ell_{i_1,j} & \text{if } \ell = \ell_{i_2,j} \text{ for some } j \in [1,m] \\ \ell_{i,j} & \text{if } \ell = \ell_{i,j} \in M \text{ and } i \notin \{i_1, i_2\} \end{cases}.$$

2. Every column of the matrix $C_j \coloneqq \{\ell_{i,j}\}_{i \in [1,n]}$ is a unique literal clause of F.

If both of these conditions are met, then we may derive the unit clauses $(\ell_{n,1})$ and $(\bar{\ell}_{i,j})$ for all $j \in [1, \min(n, m)]$ and $i \leq n - j$. In other words, we may set the bottom-left matrix entry $\ell_{n,1}$ to true and all entries in the upper-triangular portion above $\ell_{n,1}$ to false. This is possible due to Lemma 1: We can use row symmetries to swap the single satisfied literal in each ULC column of M into the lower-triangular portion of M, which leaves the upper-triangular portion of M filled with false literals.

Identifying a suitable matrix M in a given formula is non-trivial in practice, since it requires detecting row symmetries. Nevertheless, matrix symmetry is a standard notion in symmetry handling [19], and heuristic techniques for discovering such structures are common in practical symmetry-breaking systems [4, 9, 18]. Once a matrix with row symmetry is found, checking whether each column is a ULC of F is straightforward, especially since the set of ULCs in a formula can be computed in linear time [33].

3 Schreier-Sims Table Cuts

A popular technique for symmetry handling is deriving *Schreier-Sims table cuts* [1, 18, 24, 29]. In this section, we adapt this technique to iterative symmetry breaking for SAT, and we show that it is correct and that it admits short SR proofs.

Let us start with its definition. Schreier-Sims table cuts are binary clauses systematically derived from a chain of pointwise stabilizers.

▶ **Definition 3** (SST cuts). *Let F be a CNF formula, and let $\Gamma \leq \mathrm{Aut}(F)$ be a subgroup of the symmetries of F. For a literal $\ell \in \mathrm{Lit}(F)$, the* Schreier–Sims table cuts *(SST cuts) of F at ℓ are the set of binary clauses:*

$$\mathrm{SST}_\Gamma(\ell) \coloneqq \{(\overline{\ell} \vee \ell') \mid \ell' \in O_\Gamma(\ell), \ell' \neq \ell\}.$$

Intuitively, the clauses in $\mathrm{SST}_\Gamma(\ell)$ enforce that if ℓ is set to true, then all literals in its orbit must also be set to true. In other words, ℓ is the smallest literal in its orbit. An analogous definition is also possible where the binary clauses $(\ell \vee \overline{\ell'})$ make ℓ the largest literal in its orbit, but unless otherwise stated, we use the definition above.

Following the proof strategy for clausal fixing due to Anders et al. [5], we prove that adding the clauses in $\mathrm{SST}_\Gamma(\ell)$ to the formula preserves satisfiability and admits an SR proof.

▶ **Theorem 4.** *Let F be a CNF formula and $\Gamma \leq \mathrm{Aut}(F)$. Then the following hold:*

1. *F and $F \wedge \mathrm{SST}_\Gamma(\ell)$ are equisatisfiable.*
2. *There is an SR proof deriving $F \wedge \mathrm{SST}_\Gamma(\ell)$ from F in $|\mathrm{SST}_\Gamma(\ell)|$ steps.*

Proof. **(1).** Let τ be an assignment satisfying F but not $F \wedge \mathrm{SST}_\Gamma(\ell)$. Then there is a clause $\{\overline{\ell}, \ell'\} \in \mathrm{SST}_\Gamma(\ell)$ which is falsified by τ, meaning τ assigns ℓ to true and ℓ' to false.

By the definition of Schreier cuts, $\ell' \in O_\Gamma(\ell)$ holds. Hence, there exists a symmetry σ with $\sigma(\ell) = \ell'$. Using this symmetry, we obtain a new assignment $\tau' = \tau \circ \sigma$ that sets ℓ to false. Then τ' satisfies $F \wedge \mathrm{SST}_\Gamma(\ell)$: it satisfies all clauses in $\mathrm{SST}_\Gamma(\ell)$, since each clause contains $\overline{\ell}$, and it satisfies F, since σ is a syntactic symmetry of F and τ satisfies F.

(2). The SR proof derives in an arbitrary order the clauses $\{\overline{\ell}, \ell'\}$ with $\ell' \in O_\Gamma(\ell)$ and $\ell' \neq \ell$. Each clause $\{\overline{\ell}, \ell'\}$ has as witness the symmetry $\omega \in \Gamma$ mapping $\omega(\ell) = \ell'$. Suppose that L is the set of SST clauses we have already added. Then the next clause $\{\overline{\ell}, \ell'\}$ is SR if:

$$F \wedge L \wedge \{\ell\} \wedge \{\overline{\ell}'\} \vdash_1 (F \wedge L \wedge \{\overline{\ell}, \ell'\})|_\omega.$$

This is simple to show. Every clause in F is entailed, since ω is a symmetry of F, and every clause $C \in L \cup \{\overline{\ell}, \ell'\}$ is entailed by the unit clause $\{\overline{\ell}'\}$, since $\overline{\ell} \in C$ and $\omega(\overline{\ell}) = \overline{\ell}'$. ◀

SST cuts are usually added in multiple rounds: they are added for a literal ℓ, then the symmetries Γ are reduced to the pointwise stabilizer $\Gamma_{(\ell)}$ and the process is repeated. Intuitively, this process constructs a lexicographic ordering on the literals, where each new literal ℓ is placed after all previously fixed literals but before all other remaining literals.

However, because SR proofs have no notion of lexicographic order, we must use a different argument to prove that restricting the symmetries to $\Gamma_{(\ell)}$ allows us to iteratively apply SST cuts while still generating SR proofs. Fortunately, stabilizing ℓ yields *syntactic symmetries* of the newly enriched formula. Thus, the symmetries in $\Gamma_{(\ell)}$ continue to be valid SR witnesses in the resulting formula. We prove this formally in the following lemma.

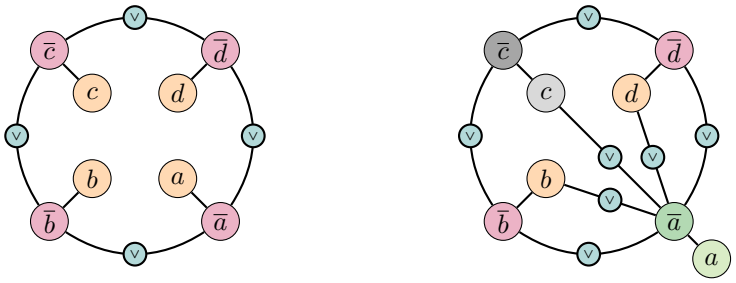


Figure 1 The variable-clause graph for the formula $(\overline{a} \vee \overline{b}) \wedge (\overline{b} \vee \overline{c}) \wedge (\overline{c} \vee \overline{d}) \wedge (\overline{d} \vee \overline{a})$ before (left) and after (right) adding SST cuts over a. The formula's symmetries are precisely the graph's geometric symmetries: the left graph has rotational and diagonal reflection symmetries, while the right graph has only one reflection symmetry. Literals with the same colors are in the same orbit.

▶ **Lemma 5.** *Let F be a CNF formula, let $\Gamma \leq \mathrm{Aut}(F)$, and let $\ell \in \mathrm{Lit}(F)$. Then:*

$$\Gamma_{(\ell)} \leq \mathrm{Aut}(F \wedge \mathrm{SST}_\Gamma(\ell)).$$

Proof. Let $\sigma \in \Gamma_{(\ell)}$. To prove that $\sigma \in \mathrm{Aut}(F \wedge \mathrm{SST}_\Gamma(\ell))$ holds, we must show that $\sigma(F \wedge \mathrm{SST}_\Gamma(\ell)) = F \wedge \mathrm{SST}_\Gamma(\ell)$. Certainly $\sigma(F) = F$, since $\sigma \in \Gamma_{(\ell)} \leq \Gamma \leq \mathrm{Aut}(F)$.

Next, we want to show that $\sigma(\mathrm{SST}_\Gamma(\ell)) = \mathrm{SST}_\Gamma(\ell)$ holds. Let $\sigma(\overline{\ell} \vee \ell') \in \sigma(\mathrm{SST}_\Gamma(\ell))$. Note that since $\sigma \in \Gamma_{(\ell)}$, we know that $\sigma(\overline{\ell}) = \overline{\ell}$ holds. By definition of SST, $\ell' \in O_\Gamma(\ell)$. Furthermore, by definition of orbits, we conclude that $\sigma(\ell') \in O_\Gamma(\ell)$ holds. Hence, we find that $\sigma(\overline{\ell} \vee \ell') = (\overline{\ell} \vee \sigma(\ell')) \in \mathrm{SST}_\Gamma(\ell)$. We can thus conclude $\sigma(\mathrm{SST}_\Gamma(\ell)) \subseteq \mathrm{SST}_\Gamma(\ell)$.

It remains to prove that each $(\overline{\ell} \vee \ell') \in \mathrm{SST}_\Gamma(\ell)$ is also contained in $\sigma(\mathrm{SST}_\Gamma(\ell))$. In order to prove this, consider the inverse of σ, which is also contained in the pointwise stabilizer of ℓ, i.e., $\sigma^{-1} \in \Gamma_{(\ell)}$. Since $\sigma^{-1}(\ell') \in O_\Gamma(\ell)$, we can easily conclude that $(\overline{\ell} \vee \sigma^{-1}(\ell')) \in \mathrm{SST}_\Gamma(\ell)$. However, this means that $(\overline{\ell} \vee \ell') = (\overline{\ell} \vee \sigma(\sigma^{-1}(\ell')) \in \sigma(\mathrm{SST}_\Gamma(\ell))$.

Finally, observe that if a clause $(\overline{\ell} \vee \ell')$ appears in both F and $\mathrm{SST}_\Gamma(\ell)$, then the combined formula $F \wedge \mathrm{SST}_\Gamma(\ell)$, by definition, only contains *one* copy of this clause. However, for any $\sigma \in \Gamma_{(\ell)}$, whenever $(\overline{\ell} \vee \ell') \in F$ holds, $(\overline{\ell} \vee \sigma(\ell')) \in F$ also holds, meaning we equally remove duplicates for all literals in the orbit of ℓ'. Hence, the claim follows. ◀

Lemma 5 allows us to derive SST cuts and their supporting SR proofs by taking Γ for Theorem 4 to be the pointwise stabilizer $\Gamma_{(\ell)}$ from the previous round. As an example, Figure 1 illustrates adding SST cuts over a particular literal a. Observe that the resulting formula retains the symmetries $\Gamma_{(a)}$. In particular, the symmetry swapping b and d remains.

Surprisingly, if we use the alternate version of Definition 3 where ℓ is the largest literal in its orbit, then Theorem 4 generalizes *clausal fixing* and *negation fixing* from prior work [5]. For clausal fixing, whenever there is a clause $C \in F$ such that $C \subseteq O_\Gamma(\ell)$, we fix ℓ to true, since satisfying C means setting its largest literal to true. For negation fixing, whenever $\overline{\ell} \in O_\Gamma(\ell)$, we fix ℓ to true to ensure that it is larger than its negation.

While Theorem 4 establishes that adding certain unit and binary clauses preserves equisatisfiability and admits SR proofs, it does not tell us *which* clauses to add. In practice, multiple literals may be fixed at any point, and so we must decide which literal to focus on and whether to apply SST cuts or apply one of orbitopal, clausal, or negation fixing. In addition, the process is inherently non-confluent, since adding clauses may destroy symmetries and thereby remove the possibility of adding others. Thus, the choice of which literal to select and which technique to apply is critical for effective symmetry breaking. We describe our algorithm for applying these reasoning rules in the following section.

4 Iterated Preprocessing

We now describe our new preprocessing algorithm for symmetry handling in SAT. We begin by describing the high-level routine, followed by more detailed descriptions of the involved subroutines and heuristics.

4.1 Preprocessing Algorithm

Algorithm 1 outlines our core routine at a high level. We implement it in SATSUMA [4].

Algorithm 1 A high-level description of our preprocessing algorithm.

```
1  function preprocess
       Input : ≻ CNF formula F
       Output : ≺ CNF formula F′ equisatisfiable to F
                ≺ SR proof
2      while true do
3          // simplify and enrich F
4          propagate unit and pure literals in F;
5          merge equivalent literals in F;
6          remove duplicate literals and duplicate/tautological clauses in F;
7          add AMO binary clauses over the ULCs in F;
8          // compute symmetries of F
9          Γ := Aut(F);
10         if |Γ| = 1 then return F;
11         // apply symmetry-handling routines
12         (F, Γ) := orbitopal-fixing(F, Γ);
13         (F, Γ) := negation-fixing(F, Γ);
14         (F, Γ) := fix-and-cut(F, Γ);
```

Description. The algorithm begins by simplifying the formula F. First, it propagates unit and pure literals (i.e., literals whose negations do not appear in F). Next, pairs of *equivalent literals* x and y are merged, where x and y are *equivalent* if they imply each other, meaning $(\overline{x} \vee y), (x \vee \overline{y}) \in F$. All occurrences of x and y in F are replaced with a canonical representative. Then, all tautological clauses, duplicate clauses, and duplicate literals in clauses are removed. Finally, AMO binary clauses are added for all ULC clauses in F.

Once the formula F is simplified, the symmetries of F are computed. If F contains no nontrivial symmetries, the algorithm terminates. Otherwise, symmetry-handling routines are applied in the order of **(1)** orbitopal fixing (Section 4.3), **(2)** negation fixing [5], and **(3)** a fix-and-cut routine applying Schreier-Sims table cuts and clausal fixing [5] (Section 4.2). Orbitopal fixing is applied first, since subsequent routines may destroy the conditions required for its application. The algorithm loops from there, alternating between formula simplification and symmetry breaking until termination or until practical limits are reached.

Regarding the choice of CNF preprocessing techniques, any equisatisfiable preprocessing method could be incorporated into the algorithm. However, because the algorithm exploits symmetries after preprocessing, it is preferable to use techniques that preserve as much useful symmetry as possible. Indeed, standard preprocessing methods such as bounded variable elimination and self-subsumption can destroy useful symmetries [2].

Correctness. The algorithm maintains the invariants that F is equisatisfiable to the original input formula and that $\Gamma \leq \mathrm{Aut}(F)$ holds. Note that since F and Γ may change throughout the execution of the algorithm, these invariants must be maintained at each step. We argue later that the orbitopal and fix-and-cut subroutines maintain these invariants.

Implementation. Similar to prior work [5], orbitopal fixing is applied using the structure detection of SATSUMA, which means the symmetries are not computed all at once. Rather, for the purpose of orbitopal fixing, only the relevant structured symmetries are computed, while the remaining symmetries are computed afterwards. This leads to a small optimization: if orbitopal fixing adds unit clauses to the formula, we can immediately restart the loop and simplify the formula again (since, after orbitopal fixing, we have not yet computed any symmetries not involved in the structures used for orbitopal fixing).

Furthermore, in practice we limit the maximum number of iterations, where larger formulas F have smaller iteration budgets. However, this limit depends on the current size of F, so if F shrinks during execution, the allowed number of iterations increases.

4.2 Fix-and-Cut

We now describe the fix-and-cut routine that is responsible in practice for many of the symmetry-breaking clauses introduced by Algorithm 1. The routine is outlined in Algorithm 2.

Algorithm 2 The fix-and-cut algorithm, which is a subroutine of Algorithm 1.

function *fix-and-cut*
  Input : ➢ CNF formula F
    ➢ permutation group $\Gamma \leq \mathrm{Aut}(F)$
  Output : ➪ CNF formula F' equisatisfiable to F
    ➪ permutation group $\Gamma' \leq \Gamma$

  $S \coloneqq \varnothing$;
  while *true* **do**
    if $|\Gamma| = 1$ **then return** (F, Γ);
    `// find candidate literal` ℓ
    if *clausal fixing can be applied to an AMO clause* C **then**
      let ℓ be some literal in C;
    else
      find a candidate literal ℓ with $|O_\Gamma(\ell)| > 1$ maximizing
       $|\{C \in F \mid \mathrm{var}(\ell) \in \mathrm{Var}(C)\} \cap \{C \in F \mid \mathrm{Var}(C) \cap \mathrm{Var}(S) \neq \emptyset\}|$
    `// fix or cut` ℓ
    if *there is a clause* $\ell \in C$ *with* $C \subseteq O_\Gamma(\ell)$ **then**
      $F \coloneqq F \wedge \{\ell\}$; `// fix` ℓ
    else
      $F \coloneqq F \wedge \mathrm{SST}_\Gamma(\ell)$; `// SST cuts over` ℓ
    `// stabilize` ℓ
    $\Gamma \coloneqq \Gamma_{(\ell)}$;
    $S \coloneqq S \cup \{\ell\}$;

Description. Given a formula F and a subgroup of its symmetries $\Gamma \leq \mathrm{Aut}(F)$, the procedure iteratively adds symmetry-breaking clauses to F using SST cuts and clausal fixing. The algorithm maintains a set S of variables that have already been fixed or used for SST cuts. It repeats the following until Γ becomes trivial (i.e., $|\Gamma| = 1$).

In each iteration, the algorithm chooses a literal ℓ that breaks the most symmetries. First, it looks for an AMO clause C that admits clausal fixing, meaning that $C \subseteq O_\Gamma(\ell)$ for some $\ell \in C$. If such a clause exists, then the algorithm fixes an arbitrary literal $\ell \in C$ to true, which causes all other literals in C to be set to false. If no such AMO clause exists, then the algorithm selects a candidate literal ℓ with nontrivial orbit that has the highest number of *shared clauses* $C \in F$ with $\mathrm{var}(\ell) \in \mathrm{Var}(C)$ and $\mathrm{var}(\ell') \in \mathrm{Var}(C)$ for some $\ell' \in S$. (In practice, if multiple literals have the same number of shared clauses, the algorithm prefers literals that admit clausal fixing.) Intuitively, literals with many shared clauses are often closely related in the formula, so focusing on them tends to break symmetries in the same part of the problem. In Section 5.2, we experimentally study the effects of using this order.

Once a candidate literal ℓ is selected, the algorithm tries to apply clausal fixing. If there exists a clause C with $\ell \in C$ such that $C \subseteq O_\Gamma(\ell)$, then the algorithm adds the unit clause $\{\ell\}$. Otherwise, the algorithm adds SST cuts for ℓ.

After adding either unit clauses or SST cuts, the subgroup Γ is updated to the pointwise stabilizer $\Gamma_{(\ell)}$, ℓ is recorded in S, and the loop repeats until termination.

Correctness. All clause additions are valid due to Theorem 4, and the update of Γ to $\Gamma_{(\ell)} \leq \mathrm{Aut}(F')$ is valid due to Lemma 5. In other words, the resulting formula F' is equisatisfiable to F, and the resulting group of symmetries Γ is a subgroup of $\mathrm{Aut}(F')$.

Implementation. The pointwise stabilizers are implemented using a random Schreier-Sims algorithm, as provided by the DEJAVU library [6, 7]. In practice, this routine is subject to a modest size limit and is invoked only when the domain size (i.e., the number of involved literals) of Γ is at most 100 000. Furthermore, the Schreier-Sims algorithm is applied with a heuristic cost limit based on the approximate number of operations performed. As a fallback mechanism, we use a cheap filtering heuristic.

The random Schreier-Sims algorithm also helps produce the SR proofs. The algorithm maintains a so-called transversal for each stabilized literal ℓ. Conceptually, the *transversal* associates each literal ℓ' in the orbit of ℓ with a symmetry σ such that $\sigma(\ell') = \ell$. (In practice, these symmetries are represented implicitly via Schreier vectors [32].) The required SR witnesses are extracted directly from these transversals.

However, there's a delicate detail when using the *random* Schreier-Sims algorithm to produce SR proofs. The algorithm may, due to randomness, return orbits that are *smaller* than the actual orbit of some literal ℓ. These erroneous orbits might get repaired in subsequent calls to the algorithm (perhaps when fixing another literal ℓ'), but this means that when we fix ℓ, we might not add all of the necessary binary clauses for ℓ. While this can never lead to deriving *incorrect* clauses, the SR proof may lack some of the clauses necessary for iterated SST cut applications, as required by Lemma 5.

We avoid this issue by implementing a two-pass algorithm: We first decide which fix-and-cut routines are going to be applied, and then we use one *consistent* call to the Schreier-Sims algorithm to produce the actual constraints and SR proof.

We end with two small implementation notes. First, rather than simplifying the formula after each loop, we perform the overlap test $C \subseteq O_\Gamma(\ell)$ on Line 11 with C reduced under S. Second, the implementation of negation fixing in Algorithm 1 is a special case of Algorithm 2. The algorithm simply chooses literals ℓ with $\overline{\ell} \in O_\Gamma(\ell)$ until no more remain.

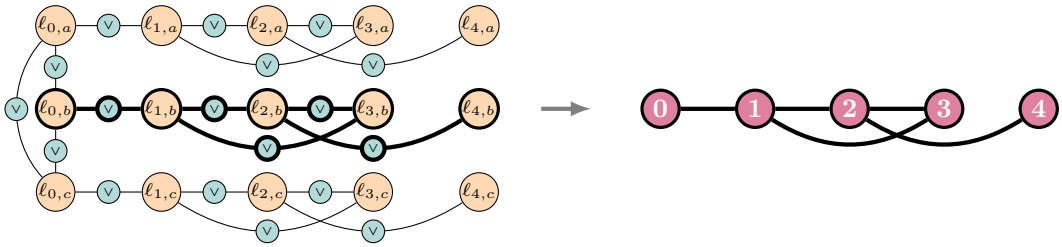


Figure 2 Extracting a graph (right) from the row symmetry of the formula (left). Intuitively, the graph only consists of one of the rows of the formula.

4.3 Clique-guided Orbitopal Fixing

Lastly, we describe a heuristic optimization for the orbitopal fixing rule (Section 2.3). Assume we can apply orbitopal fixing to a matrix M. Since the rows of M are fully symmetric, their ordering is of no importance to the algorithm. However, the *columns* of M need not be symmetric to each other, meaning they may be structurally distinct and encode vastly different things in the underlying formula. This matters because, in orbitopal fixing, all literals of the first column of M are fixed, all but two literals of the second column are fixed, all but three literals of the third column are fixed, and so on. Thus, the ordering of the columns may significantly affect how powerful the symmetry breaking is.

Given this observation, our heuristic reorders the columns of M based on finding cliques in the graph extracted from M and the formula F. Our motivation stems from *graph coloring*: If the number of colors (i.e., rows) is smaller than the size of the clique, then by moving the clique's columns to the front of M and applying orbitopal fixing, we immediately solve the graph coloring instance (i.e., orbitopal fixing propagates to conflict on the clique).

Row Connectivity Graph. Given an $n \times m$ matrix of literals M whose columns are clauses in a CNF F, we extract a graph $G(F, M)$ (or just G, if F and M are clear) as follows. Choose an arbitrary row $(\ell_1, \ldots, \ell_m)$ of M. The set of vertices is simply the set of literals in that row: $V(G) = \{\ell_1, \ldots, \ell_m\}$. Two vertices are adjacent if their corresponding literals ever occur together in a clause in F, meaning:

$$E(G) = \{\{\ell_i, \ell_j\} \mid i, j \in [1, m],\ i \neq j,\ \exists C \in F : \{\ell_i, \ell_j\} \subseteq C\}.$$

An illustration of this construction is shown in Figure 2. The formula depicted in the figure consists of the following clauses:

$$\begin{aligned} &(\ell_{0,x} \vee \ell_{1,x}) \wedge (\ell_{1,x}, \ell_{2,x}) \wedge (\ell_{2,x}, \ell_{3,x}) \wedge (\ell_{1,x}, \ell_{3,x}) \wedge (\ell_{2,x}, \ell_{4,x}) && \text{for } x \in \{a, b, c\}, \\ &(\ell_{0,x} \vee \ell_{0,y}) && \text{for } x \neq y \in \{a, b, c\}. \end{aligned}$$

Reordering Columns. Next, let $\{\ell_{i_1}, \ldots, \ell_{i_k}\} = Q \subseteq V(G)$ be a clique of G, and reorder the columns of M so that the columns containing the vertices in Q come first. Formally, we swap column j with i_j for $j \in [1, k]$. This algorithm is described in Algorithm 3.

In practice, we use the maximum-clique solver CLIQUER [38] to compute the cliques. Since the graphs only involve the variables in a single row of the matrix, the graphs tend to be much smaller than the original CNF instance, which makes the calls to CLIQUER tractable. However, we still employ practical limits: we only run CLIQUER if the initial row has fewer than 10 000 literals, and we also add an internal cost limit to CLIQUER itself. If the initial row is too large, then, starting from the first column, we greedily choose the i-th column to be a literal with the most number of shared clauses with the literals in columns $[1, i-1]$.

Algorithm 3 The orbitopal fixing algorithm with reordering based on cliques.

```
function orbitopal-fixing
    Input : ➢ CNF formula F
            ➢ permutation group Γ ≤ Aut(F)
    Output : ≺ CNF formula F′ equisatisfiable to F
             ≺ permutation group Γ′ ≤ Γ
    while true do
        // find orbitopal fixing M
        find a possible orbitopal fixing matrix M in F, Γ;
        if there is no such matrix M then return (F, Γ);
        // reorder M based on cliques
        compute graph G(F, M);
        compute maximum clique {ℓ_{i_1}, ..., ℓ_{i_k}} in G(F, M);
        swap column j with i_j in M for j ∈ [1, k];
        // apply orbitopal fixing (Section 2.3) and stabilize M
        let L be the orbitopal fixing of M;
        F ≔ F ∧ {{ℓ} | ℓ ∈ L};
        Γ ≔ Γ_(L);
```

5 Experimental Results

We implemented our techniques in SATSUMA[2] and evaluated them on three benchmark families: (1) the anniversary track of the SAT Competition 2022 [8], comprising 5344 distinct, non-random instances from the SAT Competitions 2002 to 2021, (2) the main track of the SAT Competition 2025, comprising 386 formulas, and (3) a collection of highly symmetric synthetic instances that have appeared in various papers on symmetry breaking, comprising 236 formulas representing Tseitin graphs, clique and graph coloring problems, instances from Ramsey theory, counting principle problems, channel routing problems, and several variants of the pigeonhole problem. We generated many of them using CNFGEN [28], and the two pigeons per hole (`tph`) family is due to Biere [10]. For the anniversary and 2025 suites, we filtered out all formulas that were trivially unsatisfiable (i.e., formulas containing the empty clause) and all formulas larger than 1 GB in size, since these formulas exceed internal computational limits in SATSUMA, meaning SATSUMA would not attempt to symmetry break them anyway. Removing these instances also prevents memouts while running the experiments.

We compare our new approach ("Full") against various other configurations, namely:

- SATSUMA using lex-leader constraints ("Lex-leader"),
- the version of SATSUMA used in previous work on orbitopal fixing [5] ("Orbitopal"),
- the BREAKID lex-leader symmetry-breaking tool [18] ("BREAKID"),
- and using no symmetry breaking at all ("CADICAL").

For the underlying SAT solver in all configurations, we used CADICAL[3] [11].

[2] The code is available at `https://github.com/markusa4/satsuma` and archived at `https://doi.org/10.5281/zenodo.20185023`.

[3] `https://github.com/arminbiere/cadical`. We used version 3.0.0.

We ran our experiments on the Bridges-2 cluster at the Pittsburgh Supercomputing Center [14]. Each machine has 128 cores and 256 GB of RAM. We ran our jobs in parallel across all cores, and we ran every tool with a 5000 second timeout. Notably, SATSUMA itself never ran into the 5000 second timeout.

For proof checking, we used the DSR-TRIM[4] SR proof-checking tool [16]. For a first pass, we ran every configuration with proof logging enabled in both SATSUMA and CaDiCaL, but we didn't check any proofs. Afterwards, we re-ran the Full configuration and checked its SR proofs and the DRAT proofs produced by CaDiCaL. All proofs were accepted by DSR-TRIM, aside from any timeouts and memouts.

5.1 Main Results

The summary statistics can be found in Table 1. Overall, our new iterated algorithm has the best PAR-2 scores across all three benchmark suites, and it also performs the best on many synthetic sub-families. For all variations of SATSUMA, average preprocessing time is less than 1% of the average solve time. Looking inside the competition suites: On the SAT Competition 2025, our algorithm wins over base CaDiCaL on instances from Ramsey theory and scheduling problems, as well as crafted benchmarks from clique coloring, pigeonhole, and Tseitin. On the anniversary track, many instances benefit from symmetry breaking, most of which come from the various crafted families, including many formulas in shuffled form.

On the synthetic benchmarks, several patterns emerge. First, the lex-leader configuration is consistently slower than the other variants on Tseitin instances. This gap is likely not fundamental: a good implementation of SST cuts in lex-leader should close the gap. Second, the reordering techniques that are crucial for the graph coloring instances could be transferred to lex-leader as well. We expect that these improvements would also improve runtime on the anniversary track and the SAT Competition 2025 benchmarks.

At the same time, only the Full configuration succeeds on two pigeons per hole, relativized PHP, and counting principle instances. We do not yet know how to transfer this success to lex-leader. Similarly, lex-leader performs better on binary PHP and Ramsey instances, where it is also not clear how to transfer the underlying advantages to the iterated setting.

A set of cumulative solved formula (CSF) plots in Figure 3 and Figure 4 show how many instances each configuration solves over time.

Propagation. Table 2 reports the number of instances fully propagated to UNSAT by Algorithm 1. The table also includes additional configurations from our ablation study described in Section 5.2.

Overall, we observe that our algorithm completely solves a supermajority of synthetic benchmarks, as well as a substantial number of benchmarks from the SAT Competition 2025 and the anniversary track. For many of the synthetic families, the algorithm appears to solve the instances systematically. Interestingly, this also includes families that are challenging for lex-leader, such as relativized pigeonhole principle, two pigeons per hole, and counting principle formulas. However, in the case of the latter two families, the algorithm requires many iterations, and for larger instances the implementation starts to run into our imposed practical limits on the number of iterations. As a result, not all instances from these families are solved in our experiments. We leave the fine-tuning of our practical limits to future work.

[4] `https://github.com/ccodel/dsr-trim`

Table 1 Average PAR-2 scores and number of solved formulas (#S) for each benchmark and solver. Runtimes are calculated as the sum of the SATSUMA and CaDiCaL runtimes for each formula, with a timeout of 5000 seconds. The benchmarks below "Synthetic" are the sub-families that comprise it.

Solver	Full		Orbitopal [5]		Lex-leader		BreakID [18]		CaDiCaL	
Benchmark	PAR2	#S	PAR2	#S	PAR2	#S	PAR2	#S	PAR2	#S
Anniversary	**2512.7**	**4170**	2669.4	4088	2605.6	4128	2557.0	4151	2884.4	3979
SATComp25	**2124.1**	**325**	2399.4	316	2278.8	320	2622.0	306	2737.6	301
Synthetic	**911.6**	**215**	4469.0	132	3469.6	156	4297.4	136	7110.9	70
PHP	4.0	**10**	**3.8**	**10**	6.9	**10**	5016.3	5	10000.0	0
Binary PHP	6102.4	6	8006.5	3	4680.2	8	**3362.1**	**10**	8027.9	3
Relativized PHP	**4.2**	**29**	9310.4	2	4960.4	15	7608.1	7	9656.9	1
2 Pigeons/Hole	**4003.0**	**6**	9000.0	1	6295.1	4	5007.1	5	9000.0	1
Channel	**0.2**	**10**	7.0	**10**	0.6	**10**	11.2	**10**	8005.8	2
Clique	**0.1**	**27**	0.1	**27**	0.2	**27**	1892.8	23	6361.0	11
Coloring	**369.6**	**53**	4345.1	32	4684.9	30	5273.2	26	6008.7	22
Count	**494.2**	**39**	6472.0	15	3979.1	25	6133.9	16	6646.2	14
Ramsey	4280.9	6	5015.1	5	**3045.4**	**7**	5015.3	5	5001.6	5
Tseitin	**0.1**	**20**	0.1	**20**	3023.6	14	0.1	**20**	7069.3	6
Urquhart	**0.1**	**6**	3333.4	4	5000.3	3	0.2	**6**	6919.4	2
vdW	< 0.1	**3**	< 0.1	**3**	< 0.1	**3**	< 0.1	**3**	$<$ **0.1**	**3**

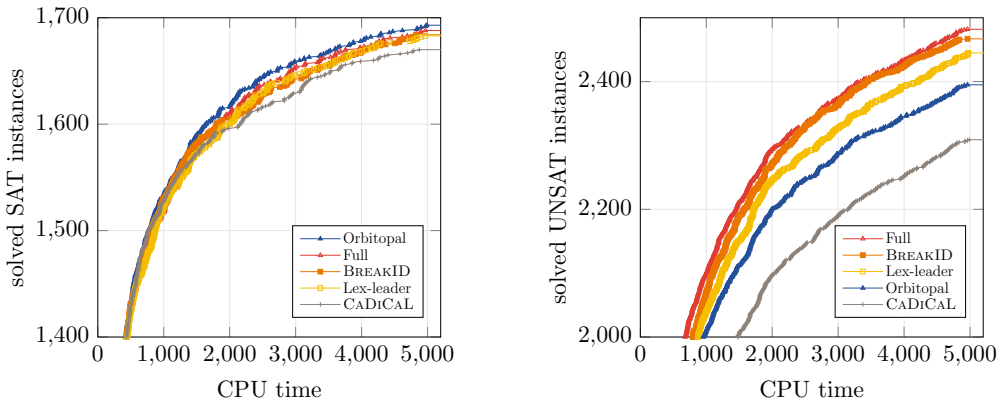


Figure 3 CSF plots on the SAT (left) and UNSAT (right) formulas of the Anniversary suite.

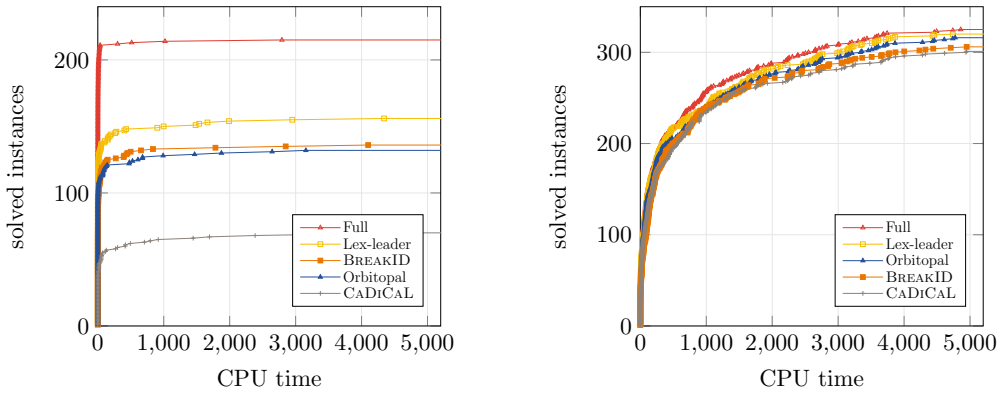


Figure 4 CSF plots on the synthetic (left) and SAT Competition 2025 (right) suites.

Table 2 The number and percentage of formulas propagated to UNSAT by each solver. The benchmarks below "Synthetic" are the sub-families that comprise it.

Solver	Full		No F&C order		No orb. order		No order	
Benchmark	#	%	#	%	#	%	#	%
Anniversary	**299**	**5.6**	283	5.3	287	5.4	271	5.1
SATComp25	**37**	**9.6**	34	8.8	**37**	**9.6**	34	8.8
Synthetic	**193**	**81.8**	158	66.9	148	62.7	113	47.9
PHP	**10**	**100.0**	**10**	**100.0**	**10**	**100.0**	**10**	**100.0**
Binary PHP	0	0	0	0	0	0	0	0
Relativized PHP	**29**	**100.0**	**29**	**100.0**	**29**	**100.0**	**29**	**100.0**
2 Pigeons/Hole	**6**	**60.0**	0	0	**6**	**60.0**	0	0
Channel	**10**	**100.0**	**10**	**100.0**	**10**	**100.0**	**10**	**100.0**
Clique	**27**	**100.0**	**27**	**100.0**	**27**	**100.0**	**27**	**100.0**
Coloring	**46**	**83.6**	**46**	**83.6**	1	1.8	1	1.8
Count	**39**	**95.1**	10	24.4	**39**	**95.1**	10	24.4
Ramsey	0	0	0	0	0	0	0	0
Tseitin	**20**	**100.0**	**20**	**100.0**	**20**	**100.0**	**20**	**100.0**
Urquhart	**6**	**100.0**	**6**	**100.0**	**6**	**100.0**	**6**	**100.0**
vdW	0	0	0	0	0	0	0	0

5.2 Ablation Study

We further conducted an ablation study. Specifically, we wanted to study:

1. the effect of repeated iteration (Algorithm 1),
2. the ordering heuristic in the fix-and-cut routine (Algorithm 2), and
3. the ordering heuristic for orbitopal fixing (Algorithm 3).

To test (1), we stop after one iteration of Algorithm 1 ("No iteration"). For (2), instead of choosing a literal ℓ as in Lines 5-9 in Algorithm 2, we choose the last literal ℓ which is contained in a nontrivial orbit of Γ ("No F&C order"). To test (3), we do not reorder matrices used for orbitopal fixing, as described in Algorithm 3 ("No orb. order"). We also study the combined effects of omitting both of these ordering heuristics ("No order").

The results of our ablation study are summarized in Table 2 and Table 3. With one exception, all ablation configurations are slower than the full configuration on the anniversary track, the SAT competition 2025, and the synthetic benchmark suite.

On the synthetic families, we observe that, as expected, orbitopal ordering is most effective on graph coloring instances, being crucial to propagate many of them to UNSAT. Iteration as well as fix-and-cut ordering seem very effective on two pigeons per hole and counting principle instances. Iteration, even without fix-and-cut ordering, seems effective on the relativized pigeonhole instances.

5.3 Proof Checking

As previously mentioned, we also checked the proofs generated by the Full configuration. Proof checking proceeded in two stages. First, we used DSR-TRIM and its sister checker LSR-CHECK to check the SR proofs generated by SATSUMA. In general, clausal proofs have *unhinted* and *hinted* versions, where unit propagation hints make proof checking take time linear in the size of the proof. This style of hinting happens for RAT (DRAT/LRAT) and for

Table 3 Average PAR-2 scores and the number of solved formulas (#S) for each benchmark family and solver. The best solver for each benchmark is bolded. Runtimes are calculated as the sum of the SATSUMA and CaDiCaL runtimes for the formula, with a timeout of 5000 seconds. The benchmarks below "Synthetic" are the sub-families that comprise it.

Solver	Full		No iteration		No F&C order		No orb. order		No order	
Benchmark	PAR2	#S	PAR2	#S	PAR2	#S	PAR2	#S	PAR2	#S
Anniversary	**2512.7**	**4170**	2576.3	4138	2548.2	4151	2526.5	4163	2563.6	4143
SATComp 25	2124.1	**325**	2185.6	323	2137.4	324	**2121.9**	**325**	2137.6	324
Synthetic	**911.6**	**215**	3064.4	165	1779.1	195	1700.4	198	2584.6	177
PHP	4.0	**10**	6.0	**10**	**3.6**	**10**	3.7	**10**	3.7	**10**
Binary PHP	6102.4	**6**	6118.9	**6**	**6056.8**	**6**	6102.4	**6**	6056.8	**6**
Relativized PHP	4.2	**29**	9310.4	2	**3.7**	**29**	3.7	**29**	3.8	**29**
2 Pigeons/Hole	4003.0	**6**	9000.0	1	8009.8	2	**4002.6**	**6**	8009.8	2
Channel	0.2	**10**	11.7	**10**	0.2	**10**	**0.2**	**10**	0.2	**10**
Clique	**0.1**	**27**	0.1	**27**	0.1	**27**	0.1	**27**	0.1	**27**
Coloring	369.6	53	**249.3**	**54**	257.7	**54**	3754.9	36	3713.9	36
Count	494.2	**39**	4994.6	21	4745.7	22	**493.9**	**39**	4745.8	22
Ramsey	4280.9	**6**	4275.1	**6**	**4002.9**	**6**	4280.8	**6**	4003.0	**6**
Tseitin	**0.1**	**20**	0.1	**20**	0.1	**20**	0.1	**20**	0.1	**20**
Urquhart	**0.1**	**6**	1666.8	5	0.1	**6**	0.1	**6**	0.1	**6**
vdW	< 0.1	**3**	< 0.1	**3**	< 0.1	**3**	$\mathbf{< 0.1}$	**3**	< 0.1	**3**

SR (DSR/LSR). DSR-TRIM both checks DSR proofs and generates a corresponding LSR proof, which is then checked by LSR-CHECK. After checking the SR proofs, we ran CaDiCaL on the symmetry-broken formulas. For any UNSAT formula, we checked the proof formed by appending the DSR proof to the DRAT proof generated by CaDiCaL.

All proofs were accepted by our proof checking tools, aside from those causing timeouts and memouts. In particular, the SR proofs were accepted regardless of the satisfiability of the formula or whether CaDiCaL timed out. The proof checking times are summarized in Table 4. Overall, the numbers are pretty typical of SAT proof checking: checking the unhinted proofs (DRAT/DSR) takes much longer than checking the hinted proofs (LRAT/LSR).

6 Conclusions and Future Work

In conclusion, we introduced a new symmetry-breaking algorithm for certified SAT solving that combines orbitopal fixing, iterated Schreier-Sims table cuts, and standard CNF preprocessing. When evaluated on a wide selection of benchmarks, our algorithm achieves substantial speedup with negligible overhead. Surprisingly, it clearly outperforms prior symmetry-breaking techniques while producing succinct SR proofs. In particular, many instances are solved completely within the new algorithm.

We hope that our approach can serve as a blueprint for future developments in efficient symmetry-handling algorithms for SAT. Indeed, several interesting directions still remain:

- Incorporate more powerful symmetry-preserving [2] CNF preprocessing techniques, to be used in between rounds of symmetry breaking.
- Raise the practical limits by making the involved algorithms more efficient.
- Design and evaluate more ordering heuristics.

Table 4 Average and median proof checking times in seconds for the DSR (unhinted) and LSR (hinted) proofs and the number of accepted proofs. The far right column reports the possible number of proofs to check. All other proofs either had trivial SR portions or caused memout.

Solver	DSR-TRIM			LSR-CHECK			
Benchmark	Avg	Med	#	Avg	Med	#	#
Anniversary (SR)	37.4	0.2	4801	1.5	0.1	4514	5344
Anniversary (SR+ RAT)	741.4	8.1	1549	4.5	0.4	1420	2160
SATComp 25 (SR)	178.0	0.7	351	2.5	0.3	325	386
SATComp 25 (SR+ RAT)	1234.1	223.6	102	5.9	1.1	82	176
Synthetic (SR)	518.0	1.7	235	27.4	0.2	222	236
Synthetic (SR+ RAT)	379.7	0.5	201	16.3	0.1	189	211

- Develop a better theoretical understanding of our iterated symmetry-handling approach, and explain why it is highly effective on some instances while providing limited benefit on others.
- Try to tackle instances for which we observed orthogonal performance between lex-leader and the new approach. In particular, it would be interesting to improve SR-based techniques on instances where currently lex-leader seems systematically more effective. This includes the Ramsey instances and the binary pigeonhole principle.
- Explore techniques for detecting more hidden or obscured symmetries, for example by incorporating cardinality detection or detecting conditional symmetries [20].

References

1 Fadi A. Aloul, Igor L. Markov, and Karem A. Sakallah. Shatter: efficient symmetry-breaking for boolean satisfiability. In *Proceedings of the 40th Design Automation Conference, DAC 2003, Anaheim, CA, USA, June 2-6, 2003*, pages 836–839. ACM, 2003. doi:10.1145/775832.776042.

2 Markus Anders. SAT preprocessors and symmetry. In *25th International Conference on Theory and Applications of Satisfiability Testing, SAT 2022, August 2-5, 2022, Haifa, Israel*, volume 236 of *LIPIcs*, pages 1:1–1:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.SAT.2022.1.

3 Markus Anders, Bart Bogaerts, Benjamin Bogø, Arthur Gontier, Wietze Koops, Ciaran McCreesh, Magnus O. Myreen, Jakob Nordström, Andy Oertel, Adrian Rebola-Pardo, and Yong Kiam Tan. Faster certified symmetry breaking using orders with auxiliary variables, 2025. doi:10.48550/arXiv.2511.16637.

4 Markus Anders, Sofia Brenner, and Gaurav Rattan. Satsuma: Structure-based symmetry breaking in SAT. *J. Artif. Intell. Res.*, 85, 2026. doi:10.1613/JAIR.1.18744.

5 Markus Anders, Cayden R. Codel, and Marijn J. H. Heule. Orbitopal fixing in SAT. In Sebastian Junges and Guy Katz, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 32nd International Conference, TACAS 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11-16, 2026, Proceedings, Part I*, Lecture Notes in Computer Science, pages 89–109. Springer, 2026. doi:10.1007/978-3-032-22752-2_5.

6 Markus Anders and Pascal Schweitzer. Parallel computation of combinatorial symmetries. In *29th Annual European Symposium on Algorithms, ESA 2021, September 6-8, 2021, Lisbon, Portugal (Virtual Conference)*, volume 204 of *LIPIcs*, pages 6:1–6:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ESA.2021.6.

7 Markus Anders and Pascal Schweitzer. Search problems in trees with symmetries: Near optimal traversal strategies for individualization-refinement algorithms. In *48th International*

Colloquium on Automata, Languages, and Programming, ICALP 2021, July 12-16, 2021, Glasgow, Scotland (Virtual Conference), volume 198 of *LIPIcs*, pages 16:1–16:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ICALP.2021.16.

8 Tomas Balyo, Marijn J. H. Heule, Markus Iser, Matti Järvisalo, and Martin Suda, editors. *Proceedings of SAT Competition 2022: Solver and Benchmark Descriptions*, Department of Computer Science Series of Publications B. Helsinki Institute for Information Technology, 2022.

9 Timo Berthold and Marc E Pfetsch. Detecting orbitopal symmetries. In *Operations Research Proceedings 2008: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR) University of Augsburg, September 3-5, 2008*, pages 433–438. Springer, 2009. doi:10.1007/978-3-642-00142-0_70.

10 Armin Biere. Two pigeons per hole problem. In Adrian Balint, Anton Belov, Marijn J.H. Heule, and Matti Järvisalo, editors, *Proceedings of SAT Competition 2013: Solver and Benchmark Descriptions*, volume B, page 103, 2013.

11 Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In Arie Gurfinkel and Vijay Ganesh, editors, *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part I*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024. doi:10.1007/978-3-031-65627-9_7.

12 Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability - Second Edition*, volume 336 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2021. doi:10.3233/FAIA336.

13 Bart Bogaerts, Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. Certified dominance and symmetry breaking for combinatorial optimisation. *J. Artif. Intell. Res.*, 77:1539–1589, 2023. doi:10.1613/JAIR.1.14296.

14 Shawn T. Brown, Paola Buitrago, Edward Hanna, Sergiu Sanielevici, Robin Scibek, and Nicholas A. Nystrom. Bridges-2: A platform for rapidly-evolving and data intensive research. In *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*, PEARC '21, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3437359.3465593.

15 Sam Buss and Neil Thapen. DRAT and propagation redundancy proofs without new variables. *Logical Methods in Computer Science*, Volume 17, Issue 2, April 2021. doi:10.23638/LMCS-17(2:12)2021.

16 Cayden R. Codel, Jeremy Avigad, and Marijn J. H. Heule. Verified substitution redundancy checking. In *Formal Methods in Computer-Aided Design, FMCAD 2024, Prague, Czech Republic, October 15-18, 2024*, pages 186–196. IEEE, 2024. doi:10.34727/2024/ISBN.978-3-85448-065-5_24.

17 James M. Crawford, Matthew L. Ginsberg, Eugene M. Luks, and Amitabha Roy. Symmetry-breaking predicates for search problems. In *Proceedings of the Fifth International Conference on Principles of Knowledge Representation and Reasoning (KR'96), Cambridge, Massachusetts, USA, November 5-8, 1996*, pages 148–159. Morgan Kaufmann, 1996.

18 Jo Devriendt, Bart Bogaerts, Maurice Bruynooghe, and Marc Denecker. Improved static symmetry breaking for SAT. In *Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings*, volume 9710 of *Lecture Notes in Computer Science*, pages 104–122. Springer, 2016. doi:10.1007/978-3-319-40970-2_8.

19 Pierre Flener, Alan M. Frisch, Brahim Hnich, Zeynep Kiziltan, Ian Miguel, Justin Pearson, and Toby Walsh. Breaking row and column symmetries in matrix models. In *Principles and Practice of Constraint Programming - CP 2002, 8th International Conference, CP 2002, Ithaca, NY, USA, September 9-13, 2002, Proceedings*, volume 2470 of *Lecture Notes in Computer Science*, pages 462–476. Springer, 2002. doi:10.1007/3-540-46135-3_31.

20 Ian P. Gent, Tom W. Kelsey, Steve Linton, Iain McDonald, Ian Miguel, and Barbara M. Smith. Conditional symmetry breaking. In *Principles and Practice of Constraint Programming - CP 2005, 11th International Conference, CP 2005, Sitges, Spain, October 1-5, 2005, Proceedings*, Lecture Notes in Computer Science, pages 256–270. Springer, 2005. doi:10.1007/11564751_21.

21 Ian P. Gent, Karen E. Petrie, and Jean-François Puget. Symmetry in constraint programming. In Francesca Rossi, Peter van Beek, and Toby Walsh, editors, *Handbook of Constraint Programming*, volume 2 of *Foundations of Artificial Intelligence*, pages 329–376. Elsevier, 2006. doi:10.1016/S1574-6526(06)80014-3.

22 Stephan Gocht and Jakob Nordström. Certifying parity reasoning efficiently using pseudo-boolean proofs. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages 3768–3777. AAAI Press, 2021. doi:10.1609/AAAI.V35I5.16494.

23 Marijn J. H. Heule, Anthony Karahalios, and Willem-Jan van Hoeve. From Cliques to Colorings and Back Again. In Christine Solnon, editor, *28th International Conference on Principles and Practice of Constraint Programming (CP 2022)*, volume 235 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:10, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.CP.2022.26.

24 Annika Jäger and Marc E. Pfetsch. Structure-preserving symmetry presolving for mixed-binary linear problems. In *Integer Programming and Combinatorial Optimization*, 2026. To appear.

25 Tim Januschowski and Marc E Pfetsch. Branch-cut-and-propagate for the maximum k-colorable subgraph problem with symmetry. In *International Conference on AI and OR Techniques in Constriant Programming for Combinatorial Optimization Problems*, pages 99–116. Springer, 2011. doi:10.1007/978-3-642-21311-3_11.

26 Matti Järvisalo, Marijn J. H. Heule, and Armin Biere. Inprocessing rules. In *Automated Reasoning*, pages 355–370, 2012. doi:10.1007/978-3-642-31365-3_28.

27 Volker Kaibel, Matthias Peinhardt, and Marc E. Pfetsch. Orbitopal fixing. *Discrete Optimization*, 8(4):595–610, 2011. doi:10.1016/j.disopt.2011.07.001.

28 Massimo Lauria, Jan Elffers, Jakob Nordström, and Marc Vinyals. CNFgen: A generator of crafted benchmarks. In Serge Gaspers and Toby Walsh, editors, *Theory and Applications of Satisfiability Testing - SAT 2017 - 20th International Conference, Melbourne, VIC, Australia, August 28 - September 1, 2017, Proceedings*, volume 10491 of *Lecture Notes in Computer Science*, pages 464–473. Springer, 2017. doi:10.1007/978-3-319-66263-3_30.

29 François Margot. Pruning by isomorphism in branch-and-cut. *Mathematical Programming*, 94(1):71–90, 2002. doi:10.1007/S10107-002-0358-2.

30 Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Comput. Sci. Rev.*, 5(2):119–161, 2011. doi:10.1016/J.COSREV.2010.09.009.

31 Adrián Rebola-Pardo. Even shorter proofs without new variables. In Meena Mahajan and Friedrich Slivovsky, editors, *26th International Conference on Theory and Applications of Satisfiability Testing (SAT 2023)*, volume 271 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 22:1–22:20, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.SAT.2023.22.

32 Ákos Seress. *Permutation Group Algorithms*. Cambridge Tracts in Mathematics. Cambridge University Press, 2003. doi:10.1017/CBO9780511546549.

33 Aeacus Sheng, Joseph E. Reeves, and Marijn J. H. Heule. Reencoding unique literal clauses. In *28th International Conference on Theory and Applications of Satisfiability Testing, SAT 2025, August 12-15, 2025, Glasgow, Scotland*, volume 341 of *LIPIcs*, pages 29:1–29:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPIcs.SAT.2025.29.

34 Charles C. Sims. Computational methods in the study of permutation groups. In *Computational Problems in Abstract Algebra*, pages 169–183. Pergamon, 1970. doi:10.1016/B978-0-08-012975-4.50020-5.

35 Mate Soos, Karsten Nohl, and Claude Castelluccia. Extending SAT solvers to cryptographic problems. In Oliver Kullmann, editor, *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings*, volume 5584 of *Lecture Notes in Computer Science*, pages 244–257. Springer, 2009. doi:10.1007/978-3-642-02777-2_24.

36 Jasper van Doornmalen and Christopher Hojny. A unified framework for symmetry handling. *Mathematical Programming, Series A*, 212:217–271, 2025. doi:10.1007/s10107-024-02102-2.

37 Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In Carsten Sinz and Uwe Egly, editors, *Theory and Applications of Satisfiability Testing - SAT 2014*, volume 8561, pages 422–429. Springer International Publishing, 2014. doi:10.1007/978-3-319-09284-3_31.

38 Patric R.J. Östergård. A fast algorithm for the maximum clique problem. *Discrete Applied Mathematics*, 120(1):197–207, 2002. Special Issue devoted to the 6th Twente Workshop on Graphs and Combinatorial Optimization. doi:10.1016/S0166-218X(01)00290-6.